

DSA: Kadane, Bit Manipulation & Math - తెలుగు గైడ్ (LeetCode 150, SSE)

ఈ document చదివిన తర్వాత Kadane (max subarray), **bit manipulation** (binary తో ఆట), మరియు **math tricks** (digits, gcd, fast power) – ఈ మూడు categories మళ్ళీ నిన్ను భయపెట్టవు. ప్రతి problem కి **ఎలా ఆలోచించాలి** (naive నుండి insight వరకు), ఏ **pattern** వాడాలి, real-life analogy, clean JavaScript solution, dry run (bit problems కి binary చూపిస్తూ), complexity, గుర్తుంచుకోవాల్సినది, మరియు సాధారణ తప్పులు – అన్నీ ఉంటాయి. లక్ష్యం: "ఒకసారి చదివితే మర్చిపోకూడదు."

ముఖ్య గమనిక: నీకు DSA అన్నలు తెలియకపోతే, ముందు [DSA_00_Foundations_Telugu.md](#) చదువు – అందులో Big-O అంటే ఏమిటి, time/space complexity, array/loop basics ఉన్నాయి. ఇక్కడ మనం నేరుగా LeetCode Top-Interview-150 లోని **Kadane/Greedy + Bit Manipulation + Math** (14 problems) ని senior (SSE) interview స్థాయిలో పట్టిస్తాం. Solutions అన్నీ **JavaScript (ES2020+)** లో.

JavaScript గురించి ఒక కీలక హెచ్చరిక: JS లో bitwise operators (& , | , ^ , << , >>) అన్నీ **32-bit signed integers** మీద పని చేస్తాయి. అంటే top bit (bit 31) ఉంటే number **negative** గా చదవబడుతుంది. అందుకే bit problems లో మనం చాలాసార్లు >>> (**unsigned right shift**) వాడాలి. ఇది ఈ guide లో పదే పదే గుర్తుచేస్తాం.

విషయ సూచిక (Table of Contents)

మొదట Patterns నేర్చుకో (ఇవి 14 problems కి పునాది):

- Pattern: Kadane's Algorithm (running max subarray – "extend vs restart" నిర్ణయం)
- Pattern: Bit Manipulation (binary basics from scratch, AND/OR/XOR/shift, tricks: $n \& (n-1)$, XOR self-cancel, masks)
- Pattern: Math Tricks (digits extraction, gcd, fast exponentiation, factor counting)

Problems (LeetCode Top Interview 150 – Kadane / Bit / Math):

- Maximum Subarray (#53) – Medium
- Maximum Sum Circular Subarray (#918) – Medium
- Add Binary (#67) – Easy
- Reverse Bits (#190) – Easy
- Number of 1 Bits (#191) – Easy
- Single Number (#136) – Easy
- Single Number II (#137) – Medium
- Bitwise AND of Numbers Range (#201) – Medium
- Palindrome Number (#9) – Easy
- Plus One (#66) – Easy
- Factorial Trailing Zeroes (#172) – Medium

12. Sqrt(x) (#69) — Easy

13. Pow(x, n) (#50) — Medium

14. Max Points on a Line (#149) — Hard

Patterns (ముందు ఇవి పట్టుకో)

Interview లో success అంటే 300 problems బట్టి కొట్టడం కాదు — కొన్ని **core patterns** లోతుగా అర్థం చేసుకోవడం. కొత్త problem వచ్చినప్పుడు "ఇది ఏ pattern?" అని గుర్తుపడితే, solution దానంతట అదే బయటకు వస్తుంది. కింద ఈ 14 problems కి కావాల్సిన మూడు core patterns ఉన్నాయి — Kadane, Bit Manipulation, Math tricks.

Pattern: Kadane's Algorithm

వివరణ

Kadane's algorithm అనేది **"maximum sum contiguous subarray"** (వరుసగా ఉన్న elements లో గరిష్ట మొత్తం) ని $O(n)$ లో కనుక్కునే classic technique. Array మీద ఒక్కసారి నడుస్తూ, ప్రతి index దగ్గర ఒక్కటే నిర్ణయం తీసుకుంటాం:

"ఇక్కడిదాకా వచ్చిన subarray ని కొనసాగించాలా (extend), లేక ఇక్కడి నుండి కొత్తగా మొదలుపెట్టాలా (restart)?"

ఈ నిర్ణయం చాలా simple: మనం ఇప్పటిదాకా పోగుచేసిన మొత్తం (maxEndingHere) **negative** గా ఉంటే, అది future కి **భారం** (baggage) మాత్రమే — దాన్ని వదిలేసి ఈ element నుండి కొత్తగా మొదలుపెడితేనే మంచిది. అదే `maxEndingHere = Math.max(nums[i], maxEndingHere + nums[i])`. ఈ ఒక్క line లోనే మొత్తం algorithm ఉంది.

రెండో variable `maxSoFar` — ఇప్పటిదాకా చూసిన అన్ని subarrays లో **అత్యుత్తమమైనది** ని గుర్తుపెట్టుకుంటుంది (ఎందుకంటే best subarray ఎక్కడ ముగుస్తుందో మనకు ముందే తెలియదు).

Real-life analogy: నువ్వు ఒక road trip లో ఉన్నావు, ప్రతి town లో లాభమో నష్టమో అవుతుంది (profit/loss). నీ దగ్గర ఒక "running wallet" (maxEndingHere) ఉంది. ఒక town చేరేసరికి నీ wallet **అప్పులో (negative)** ఉంటే — ఆ అప్పుని మోసుకుని ముందుకు వెళ్ళడం మూర్ఖత్వం; wallet ని reset చేసి **ఈ town నుండి కొత్త trip** మొదలుపెట్టడం మేలు. కానీ ఇప్పటిదాకా నీవు చూసిన **అత్యధిక wallet value** (maxSoFar) ని మాత్రం మర్చిపోకుండా diary లో రాసిపెట్టుకుంటావు. ఇదే Kadane.

ఎలా గుర్తించాలి (recognition signals)

- "**Contiguous** subarray" (వరుస elements) లో "maximum/minimum sum", "maximum product" అని అడిగితే.
- Array లో positive & negative రెండూ ఉండి, "best స్పెష్ ఏది" అనే optimization అయితే.
- Brute force $O(n^2)/O(n^3)$ కనిపిస్తోంది కానీ "ఒక్క pass లో running best maintain చేయగలమా?" అనిపిస్తే.
- "Circular", "product", "at most k" లాంటి twists — ఇవి అన్నీ Kadane యొక్క variants.

Reusable Template

```
function kadane(nums) {  
  let maxEndingHere = nums[0]; // ఇక్కడ ముగిసే subarray యొక్క best sum  
  let maxSoFar = nums[0];      // ఇప్పటిదాకా చూసిన global best (answer)
```

```
for (let i = 1; i < nums.length; i++) {  
  // కేలక నిర్ణయం: పాతదాన్ని extend చెయ్యాలా, లేక ఇక్కడ restart చెయ్యాలా?  
  maxEndingHere = Math.max(nums[i], maxEndingHere + nums[i]);  
  // global best ని update చెయ్యి  
  maxSoFar = Math.max(maxSoFar, maxEndingHere);  
}  
return maxSoFar;  
}
```

ముఖ్యం: maxSoFar ని nums[0] తో initialize చెయ్యి, 0 తో కాదు. లేకపోతే array లో **అన్ని negative** అయితే (ఉదా. [-3, -1, -2]) తప్పు answer 0 వస్తుంది (సరైనది **-1**). Subarray ఖాళీగా (empty) ఉండకూడదు – కనీసం ఒక్క element ఉండాలి.

Complexity

- **Time:** $O(n)$ – array మీద ఒక్కటే pass.
 - **Space:** $O(1)$ – కేవలం రెండు variables. Extra array అవసరం లేదు.
-

Problems

1. Maximum Subarray (LeetCode #53) — Medium

- సమస్య:** ఒక integer array `nums` (positive, negative, zero కలిసి) ఇస్తారు. దానిలో **contiguous** (వరుసగా ఉన్న) non-empty subarray తీసుకుంటే, వాటి మొత్తం (sum) గరిష్టంగా ఎంత వస్తుందో ఆ **maximum sum** ని return చెయ్యాలి.
- Key constraints:** Subarray కనీసం ఒక్క element అయినా కలిగి ఉండాలి (empty కాదు). `1 <= nums.length <= 10^5`. Elements `-10^4` నుండి `10^4` వరకు.
- ఉదాహరణ:**

```
Input:  nums = [-2,1,-3,4,-1,2,1,-5,4]
Output: 6
వివరణ:  subarray [4,-1,2,1] యొక్క sum = 6 (అన్నిటిలో గరిష్టం)
```

- ఎలా ఆలోచించాలి:**

మొదటి ఆలోచన (brute force): అన్ని subarrays తీసుకుని, ప్రతి దాని sum లెక్కించి, గరిష్టం పట్టుకోవడం. Subarrays $O(n^2)$ ఉంటాయి, ప్రతి sum $O(n)$ → మొత్తం $O(n^3)$. Prefix sums వాడితే $O(n^2)$ కి తగ్గించవచ్చు. కానీ $n = 10^5$ కి $O(n^2) = 10^{10}$ operations — చాలా నెమ్మది.

ఏం గమనించాలి? ఒక subarray ని కుడివైపుకి పొడిగిస్తూ (extend) పోతున్నప్పుడు — "ఇప్పటిదాకా పోగైన మొత్తం **negative** అయితే, దాన్ని కొనసాగించడం వల్ల future కి నష్టమే." ఎందుకంటే negative prefix ఏ subarray కి తోడైనా, దాన్ని తగ్గిస్తుంది. కాబట్టి prefix negative అయిన క్షణం, దాన్ని వదిలేసి ఈ element నుండి **కొత్తగా మొదలుపెట్టాలి**. ఇదే Kadane's insight — "extend vs restart."

- Brute Force / naive:** అన్ని (i, j) జతలకి subarray sum. Prefix sum తో $O(n^2)$ time, $O(1)$ space. n పెద్దదైతే TLE (Time Limit Exceeded). ఎందుకు insufficient — same overlapping పనిని మళ్ళీ మళ్ళీ చేస్తున్నాం.
- Optimal Approach: Kadane's Algorithm.** ఒక్క pass. `maxEndingHere` = ఇక్కడ ముగిసే subarray best sum; ప్రతి element దగ్గర `Math.max(nums[i], maxEndingHere + nums[i])` తో extend-or-restart నిర్ణయం. `maxSoFar` = global best. $O(n)$ time, $O(1)$ space.
- Solution (JavaScript):**

```
/**
 * @param {number[]} nums
 * @return {number} - గరిష్ట contiguous subarray sum
 */
var maxSubArray = function (nums) {
  let maxEndingHere = nums[0]; // ఇక్కడ (index i వద్ద) ముగిసే subarray యొక్క best sum
  let maxSoFar = nums[0];      // ఇప్పటిదాకా చూసిన అన్నిటిలో గరిష్టం (final answer)

  for (let i = 1; i < nums.length; i++) {
```

```
// extend చెయ్యాలా (maxEndingHere + nums[i]) లేక restart చెయ్యాలా (nums[i] ఒక్కటే)?
// pastPrefix negative అయితే nums[i] ఒక్కటే గెలుస్తుంది → అదే restart.
maxEndingHere = Math.max(nums[i], maxEndingHere + nums[i]);

// ఈ కొత్త maxEndingHere, global best ని దాటేందా?
maxSoFar = Math.max(maxSoFar, maxEndingHere);
}

return maxSoFar;
};
```

- **Dry Run:** nums = [-2,1,-3,4,-1,2,1,-5,4] . మొదట maxEndingHere = maxSoFar = -2 .

i	nums[i]	ext = prev+nums[i]	maxEndingHere = max(nums[i], ext)	maxSoFar
1	1	-2+1=-1	max(1,-1)=1	1
2	-3	1-3=-2	max(-3,-2)=-2	1
3	4	-2+4=2	max(4,2)=4	4
4	-1	4-1=3	max(-1,3)=3	4
5	2	3+2=5	max(2,5)=5	5
6	1	5+1=6	max(1,6)=6	6
7	-5	6-5=1	max(-5,1)=1	6
8	4	1+4=5	max(4,5)=5	6

Answer maxSoFar = 6 — subarray [4,-1,2,1] . ✅ గమనించు: index 2 దగ్గర prefix -2 (negative) అయినా, index 3 లో nums[i]=4 గెలిచి restart జరిగింది.

- **Complexity:** Time $O(n)$ — ఒక్క pass, ప్రతి element ఒక్కసారే. Space $O(1)$ — రెండే variables.
- **గుర్తుంచుకోవాల్సినది:** Kadane = "running sum negative అయితే throw చెయ్యి, లేకపోతే carry చెయ్యి; అన్నిటిలో best ని diary లో రాసుకో." ఈ extend-vs-restart idea, sum బదులు **product** (Maximum Product Subarray), లేదా **circular** array (తర్వాతి problem) కి కూడా విస్తరిస్తుంది. Kadane ఒక్కసారి పట్టుకుంటే చాలా doors తెరుచుకుంటాయి.
- **సాధారణ తప్పులు:**
 - maxSoFar ని 0 తో initialize చెయ్యడం → అన్నీ negative అయితే ([-1,-2,-3]) తప్పు 0 వస్తుంది. nums[0] తో మొదలుపెట్టు.
 - maxSoFar update ని maxEndingHere update లోపల if-else లో పెట్టి మర్చిపోవడం. రెండూ ప్రతి iteration లో జరగాలి.
 - Empty subarray allowed అని పొరపడటం — ఈ problem లో కనీసం ఒక్క element తప్పనిసరి.

2. Maximum Sum Circular Subarray (LeetCode #918) – Medium

- **సమస్య:** ఒక **circular** integer array `nums` ఇస్తారు (అంటే చివరి element తర్వాత మళ్ళీ మొదటి element వస్తుంది – రింగ్ లాగా). దానిలో non-empty **contiguous** subarray యొక్క గరిష్ట sum ని return చెయ్యాలి. Circular కాబట్టి subarray array చివర్లో మొదలై, wrap అయి మొదట్లో ముగియవచ్చు.
- **Key constraints:** Subarray లో ప్రతి element **ఒక్కసారే** వాడాలి (ఒకే element ని wrap వల్ల రెండుసార్లు లెక్కించకూడదు). $1 \leq \text{nums.length} \leq 3 \cdot 10^4$.
- **ఉదాహరణ:**

Input: `nums = [5,-3,5]`

Output: `10`

వివరణ: చివరి 5 నుండి wrap అయి మొదటి 5 దాకా $\rightarrow [5, (\text{wrap}), 5] = 5+5 = 10$

- **ఎలా ఆలోచించాలి:**

Answer రెండు రకాలుగా ఉండొచ్చు:

1. **Non-wrapping** (మధ్యలో ఒక సాధారణ subarray) – దీనికి plain Kadane maximum చాలు.
2. **Wrapping** (చివర్లో కొంత + మొదట్లో కొంత, మధ్య భాగం వదిలేసి).

రెండో సందర్భంలో ఒక అందమైన trick ఉంది: మనం **తీసుకునే** భాగం wrap అయితే, మనం **వదిలేసే** భాగం (మధ్యలోది) ఒక సాధారణ non-wrapping subarray. కాబట్టి "wrap చేసి max తీసుకోవడం" = "మొత్తం sum నుండి, **minimum** non-wrapping subarray ని తీసివేయడం" = $\text{total} - \text{minSubarraySum}$.

అంటే `answer = max( kadaneMax, total - kadaneMin )`.

ఒక్క trap: అన్ని elements negative అయితే – `total - minSum` లో `minSum` మొత్తం array అవుతుంది, మిగిలేది **empty subarray (0)** – అది invalid! అలాంటప్పుడు plain `kadaneMax` (అతి తక్కువ negative) ను return చెయ్యాలి. దీన్ని `if (kadaneMax > 0)` check తో పట్టుకుంటాం (max positive అయితేనే wrap case valid).

- **Brute Force / naive:** ప్రతి starting index నుండి `n` elements దాకా wrap-around sums లెక్కించడం $\rightarrow O(n^2)$. $n = 3 \cdot 10^4$ కి $9 \cdot 10^8$ operations – నెమ్మది.
- **Optimal Approach: Kadane రెండుసార్లు** – ఒకసారి max subarray కోసం, ఒకసారి min subarray కోసం – ఒకే loop లో. Answer = `max(maxSum, total - minSum)`, కానీ `maxSum <= 0` (అన్నీ negative) అయితే `maxSum`. $O(n)$ time, $O(1)$ space.
- **Solution (JavaScript):**

```
/**
 * @param {number[]} nums
 * @return {number} - circular array లో గరిష్ట subarray sum
 */
var maxSubarraySumCircular = function (nums) {
    let total = 0; // మొత్తం array sum
    let curMax = 0, maxSum = nums[0]; // Kadane for MAX subarray
    let curMin = 0, minSum = nums[0]; // Kadane for MIN subarray
```

```

for (const n of nums) {
  // max Kadane: extend లేదా restart
  curMax = Math.max(curMax + n, n);
  maxSum = Math.max(maxSum, curMax);

  // min Kadane: అదే idea కానీ minimum కోసం
  curMin = Math.min(curMin + n, n);
  minSum = Math.min(minSum, curMin);

  total += n;
}

// maxSum > 0 అంటే కనీసం ఒక positive ఉంది → wrap case (total - minSum) valid.
// maxSum <= 0 అంటే అన్నీ negative → non-wrapping maxSum నే answer.
return maxSum > 0 ? Math.max(maxSum, total - minSum) : maxSum;
};

```

- **Dry Run:** `nums = [5, -3, 5]` , total చివర్లో = 7 .

n	curMax	maxSum	curMin	minSum
5	$\max(0+5, 5)=5$	5	$\min(0+5, 5)=5$	5
-3	$\max(5-3, -3)=2$	5	$\min(5-3, -3)=-3$	-3
5	$\max(2+5, 5)=7$	7	$\min(-3+5, 5)=2$	-3

total = 7 , minSum = -3 . maxSum = 7 > 0 → $\max(7, 7 - (-3)) = \max(7, 10) = 10$. ✓ ఇది wrap subarray [5, _, 5] .

- **Complexity:** Time $O(n)$ — ఒక్క pass లో రెండు Kadanes. Space $O(1)$.
- **గుర్తుంచుకోవాల్సినది:** "Circular లో max = సాధారణ max, లేదా (total - min)." Wrap అయ్యే భాగం ని నేరుగా లెక్కించడం కష్టం, కానీ దాని complement (వదిలేసిన మధ్య భాగం) ను minimize చేయడం సులభం. "తీసుకోవడం కష్టమైతే, వదిలేసేదాన్ని optimize చెయ్యి" — ఇది చాలా చోట్ల పనికొచ్చే ఆలోచన.
- **సాధారణ తప్పులు:**
 - **All-negative edge case** మర్చిపోవడం (ఉదా. [-3, -2, -3]). total - minSum = empty = 0 ను తప్పుగా return చెయ్యడం. maxSum > 0 guard తప్పనిసరి.
 - maxSum , minSum ని 0 తో initialize చెయ్యడం (single-element/all-negative arrays కి తప్పు). nums[0] తో మొదలుపెట్టు.
 - రెండు Kadanes కి ఒకే cur variable వాడటం — max, min కి వేరువేరు curMax , curMin కావాలి.

Pattern: Bit Manipulation

వివరణ — Binary ని scratch నుండి

Computer లో ప్రతి number **binary** (0లు, 1లు) గా నిల్వ ఉంటుంది. మనం రోజూ వాడే decimal (base-10) లో ప్రతి digit place value ...**100**, **10**, **1** (powers of 10). Binary (base-2) లో place values **powers of 2**: ... **16**, **8**, **4**, **2**, **1**. ప్రతి స్థానాన్ని **bit** అంటారు.

ఉదా. 13 ని binary గా చూద్దాం:

place value: 8 4 2 1
bit: 1 1 0 1 → 8 + 4 + 0 + 1 = 13
అంటే 13 = binary 1101

కొన్ని ముఖ్యమైనవి:

Decimal	Binary	Note
0	0000	అన్నీ 0
1	0001	lowest bit (bit 0)
2	0010	bit 1
4	0100	bit 2
5	0101	4+1
8	1000	bit 3

Bit position: కుడివైపు నుండి 0 తో మొదలు (bit 0 = lowest = 1s place). Bit **i** యొక్క value = 2^i .

Bitwise Operators — ఐదు ఆయుధాలు

ఇవి రెండు numbers ని **bit-by-bit** గా compare/మార్పు చేస్తాయి:

1. AND (&) — రెండు bits **రెండూ 1** అయితేనే 1.

5 = 0101
6 = 0110
5&6 = 0100 = 4 (కేవలం bit 2 లో రెండూ 1)

2. OR (|) — కనీసం ఒకటి 1 అయితే 1.

5|6 = 0111 = 7

3. XOR (^) — సరిగ్గా ఒకటి 1 అయితే 1 (రెండూ ఒకేలా ఉంటే 0). "different అయితే 1."

5^6 = 0011 = 3

4. **NOT (~)** – అన్ని bits flip ($0 \leftrightarrow 1$). JS లో 32-bit signed కాబట్టి $\sim n === -(n+1)$. ఉదా. $\sim 5 = -6$.

5. Shifts:

- $n \ll k$ (left shift) = bits ని ఎడమకి k స్థానాలు జరుపు = $n \times 2^k$. $5 \ll 1 = 1010 = 10$.
- $n \gg k$ (arithmetic right shift) = కుడికి జరుపు, sign bit ని copy చేస్తూ (negative అలాగే ఉంటుంది).
- $n \ggg k$ (**unsigned** right shift) = కుడికి జరుపు, ఎడమన **0లు** నింపుతూ. Negatives ని పెద్ద positive గా చూస్తుంది.

మూడు golden tricks (interview లో పదే పదే)

Trick 1 – $n \& (n-1)$ = lowest set bit ని తీసేస్తుంది.

```
12 = 1100
11 = 1011
12 & 11 = 1000 = 8    (కుడివైపు మొదటి 1 పోయింది)
```

ఎన్నిసార్లు ఇది చేయగలమో అన్ని **set bits (1లు) ఉన్నాయి** అని అర్థం $\rightarrow$ bit counting కి golden.

Trick 2 – **XOR self-cancel:** $a \wedge a = 0$, $a \wedge 0 = a$.

XOR commutative & associative. కాబట్టి list లో ప్రతి number **జతగా (pairs)** వస్తే, అవన్నీ XOR చేస్తే **cancel** అయిపోయి, జత లేని ఒక్కటి మిగులుతుంది. "duplicate ఒకటి తప్ప అన్నీ జతలు" problems కి magic.

Trick 3 – **Masks (bit ని చదవడం/set/clear చెయ్యడం):**

```
(n >> i) & 1      // bit i విలువ (0 లేదా 1) చదువు
n | (1 << i)      // bit i ని 1 చెయ్యి (set)
n & ~(1 << i)     // bit i ని 0 చెయ్యి (clear)
n ^ (1 << i)      // bit i ని flip (toggle)
n & (-n)          // lowest set bit ని isolate చెయ్యి
```

Real-life analogy: ఒక వరుస **light switches** (each bit = ఒక switch, on=1/off=0) ఊహించుకో. **AND** = "రెండు boards లోనూ ఏ switches on ఉన్నాయి" (intersection). **OR** = "కనీసం ఒక board లో on" (union). **XOR** = "ఒకే board లో on – రెండింటిలో కాదు" (వేరైన switches). **shift** = మొత్తం panel ని ఎడమ/కుడికి జరపడం. Bit manipulation అంటే – loops తో లెక్కించే బదులు, ఈ switches ని ఒక్క operation లో సామూహికంగా మార్చడం. అందుకే ఇది చాలా **fast** (hardware level).

⚠ JavaScript 32-bit signed trap (అతి ముఖ్యం)

JS numbers అన్నీ 64-bit float. కానీ ఏ bitwise operation చేసినా, JS ముందు number ని **32-bit signed integer** గా మార్చి, తర్వాత operation చేస్తుంది. దీని వల్ల:

- Top bit (bit 31) set అయితే number negative** గా చదవబడుతుంది. ఉదా. $1 \ll 31 = -2147483648$ (positive 2^{31} కాదు!).

- Unsigned గా కావాలంటే `n >>> 0` వాడు — ఇది 32-bit unsigned గా reinterpret చేస్తుంది. `(1 << 31) >>> 0 = 2147483648`.
- Loop లో number ని shift చేస్తూ bits చదివేటప్పుడు, negative sign propagate అవ్వకుండా `>>>` (**unsigned shift**) వాడు, `>>` కాదు.

ఈ trap వల్లే bit problems లో మనం `>>>` పదే పదే వాడతాం. గుర్తుంచుకో: **"bit లెక్క లో అనుమానం వస్తే `>>>` వాడు."**

ఎలా గుర్తించాలి (recognition signals)

- Constraints లో **"O(1) space"**, "without extra memory", "constant space" — hashmap కి బదులు bits.
- "appears twice/thrice except one", "single number", "duplicate" — XOR లేదా bit counting.
- "binary representation", "number of 1 bits", "reverse bits", "power of two" — నేరుగా bit ops.
- Numbers range మీద AND/OR, subsets generate చెయ్యడం (bitmask).

Reusable Templates

```
// 1. set bits లెక్కించు (Brian Kernighan)
function countBits(n) {
    let count = 0;
    while (n !== 0) { n &= (n - 1); count++; } // lowest 1 ని తీస్తూ
    return count;
}

// 2. అన్ని 32 bits మీద నడుపు (unsigned గా చదవడానికి >>> వాడు)
for (let i = 0; i < 32; i++) {
    const bit = (n >>> i) & 1; // bit i విలువ
    // ...
}

// 3. XOR అన్నీ (జతలు cancel అవుతాయి)
let x = 0;
for (const num of nums) x ^= num;
```

Complexity

- Set-bit tricks & fixed 32-bit loops: **O(1)** per number (32 స్థిరం), లేదా n numbers కి **O(n)**.
- Space: దాదాపు ఎప్పుడూ **O(1)** — ఇదే bit manipulation యొక్క గొప్పతనం (hashmap అవసరం లేదు).

3. Add Binary (LeetCode #67) — Easy

- **సమస్య:** రెండు binary strings `a`, `b` ('0'/'1' అక్షరాలతో) ఇస్తారు. వాటి **మొత్తాన్ని (sum)** మళ్ళీ ఒక binary string గా return చెయ్యాలి.
- **Key constraints:** Strings చాలా పొడవుగా ఉండొచ్చు (`1 <= length <= 10^4`) — కాబట్టి integer గా parse చేస్తే **overflow** అవుతుంది (2^{453} దాటుతుంది). Leading zeros ఉండవు (single "0" తప్ప).

- ఉదాహరణ:

```
Input: a = "11", b = "1"
Output: "100"
వివరణ: 3 + 1 = 4 → binary 100
```

- ఎలా ఆలోచించాలి:

School లో decimal addition ఎలా చేశావు? కుడివైపు (చివరి digit) నుండి మొదలు, digit-by-digit కూడు, carry ని ఎడమకి తీసుకెళ్ళు. Binary లో కూడా అదే — తేడా ఒక్కటే: decimal లో 10 కి reset అయితే, binary లో **2 కి reset**. అంటే **1 + 1 = 10** (digit 0, carry 1).

ఏం గమనించాలి? Number గా convert చేసి కూడడం tempting, కానీ length $10^4 \rightarrow$ number type overflow. కాబట్టి string మీదే, చివరి నుండి, manual carry addition. మూడు bits కూడతాం: **a bit + b bit + carry . sum % 2 =** రాయాల్సిన digit, **sum >= 2** అయితే carry 1 .

- Brute Force / naive:** `parseInt(a, 2) + parseInt(b, 2)` తర్వాత `.toString(2)` . చిన్న inputs కి పని చేస్తుంది కానీ length పెరిగితే **precision loss / overflow** (JS numbers 2^{53} దాకానే safe). BigInt వాడితే పని చేస్తుంది (`(BigInt('0b'+a)+BigInt('0b'+b)).toString(2)`) కానీ interviewer manual carry logic చూడాలనుకుంటాడు.
- Optimal Approach: Two pointers from the end + carry.** **i** , **j** రెండు strings చివర్ల నుండి; ఒక carry . మూడింటినీ కూడి, `% 2` push, `carry = Math.floor(sum/2)` . చివర్లో reverse చేసి join. **$O(\max(m,n))$** time.
- Solution (JavaScript):**

```
/**
 * @param {string} a - binary string
 * @param {string} b - binary string
 * @return {string} - a + b (binary)
 */
var addBinary = function (a, b) {
    let i = a.length - 1; // a యొక్క చివరి bit
    let j = b.length - 1; // b యొక్క చివరి bit
    let carry = 0;
    const result = []; // digits ని వెనుకనుండి పోగుచేస్తాం

    // ఏ ఒక్క string మిగిలున్నా, లేదా carry మిగిలున్నా కొనసాగించు
    while (i >= 0 || j >= 0 || carry > 0) {
        let sum = carry;
        if (i >= 0) sum += a.charCodeAt(i) - 48; // '0' యొక్క code 48; '1'-48=1
        if (j >= 0) sum += b.charCodeAt(j) - 48;

        result.push(sum % 2); // ఈ స్థానంలో రాయాల్సిన digit (0 లేదా 1)
        carry = sum >= 2 ? 1 : 0; // binary → 2 తో reset

        i--;
        j--;
    }
}
```

```
return result.reverse().join(''); // వెనుకనుండి పోగుచేసాం కాబట్టి reverse
};
```

- **Dry Run:** $a = "11"$, $b = "1"$. మొదట $i=1$, $j=0$, $carry=0$.

step	a[i]	b[j]	sum = carry+a+b	push (sum%2)	new carry
1	1	1	0+1+1=2	0	1
2	1	—	1+1=2	0	1
3	—	—	1	1	0

$result = [0,0,1] \rightarrow reverse \rightarrow [1,0,0] \rightarrow "100"$. ✓ (3 + 1 = 4 = binary 100).

- **Complexity:** Time $O(\max(m, n))$ — పొడవైన string పొడవు. Space $O(\max(m, n))$ — result string కి (అది తప్పనిసరి output).
- **గుర్తుంచుకోవాల్సినది: "చివరి నుండి digit-by-digit + carry"** — ఇది Add Binary, Plus One, Add Two Numbers (linked list), string multiplication అన్నిటికీ ఒకటే pattern. Base మారుతుంది (2 vs 10), logic ఒకటే. Loop condition లో $carry > 0$ కూడా చేర్చడం మర్చిపోకు — లేకపోతే $"11" + "1"$ లో చివరి carry పోతుంది.
- **సాధారణ తప్పులు:**
 - Loop condition లో $carry > 0$ మర్చిపోవడం → చివరి carry ($"100"$ లోని leading 1) రాదు.
 - చివర్లో `reverse()` మర్చిపోవడం → answer తలకిందులుగా వస్తుంది.
 - Numbers గా parse చేసి overflow తినడం (పెద్ద inputs కి). String మీదే పని చెయ్యి.
 - `a.charCodeAt(i)` — 48 బదులు `a[i]` (string) ని నేరుగా కూడితే $"1"+"1" = "11"$ (string concat) — number గా convert చెయ్యడం మర్చిపోకు.

4. Reverse Bits (LeetCode #190) — Easy

- **సమస్య:** ఒక 32-bit unsigned integer n ఇస్తారు. దాని bits ని తలకిందులు (reverse) చేసి వచ్చే number ని return చెయ్యాలి. అంటే bit 0 ↔ bit 31, bit 1 ↔ bit 30, ... swap.
- **Key constraints:** Input సరిగ్గా 32 bits (leading zeros తో సహా లెక్క). JS లో result **unsigned** గా return చెయ్యాలి ($>>> 0$).
- **ఉదాహరణ:**

Input: $n = 43261596$ (binary: 00000010100101000001111010011100)
Output: 964176192 (binary: 00111001011110000010100101000000)
వివరణ: 32-bit pattern ని అద్దం లో చూసినట్టు తిరగేశాం.

- **ఎలా ఆలోచించాలి:**

Bits ని reverse చెయ్యడం అంటే — string ని reverse చేసినట్టే, కానీ characters బదులు bits. మనం n యొక్క **lowest bit** ని తీసి, **result** యొక్క **highest** స్థానంలో పెడితే? అలా 32 సార్లు చేస్తే — first-in bit last position కి, last-in bit

first position కి వెళ్తుంది. అదే reversal.

ప్లాన్: ప్రతి step లో – **result** ని ఎడమకి ఒక bit జరుపు ($\text{result} \ll 1$, ఖాళీ చేస్తూ), దానిలో n యొక్క lowest bit ($n \& 1$) ని పెట్టు ($|$). తర్వాత n ని కుడికి జరుపు ($n \gg= 1$). ఇక్కడ $\gg$ **తప్పనిసరి** – n unsigned కాబట్టి sign bit propagate అవకూడదు.

- **Brute Force / naive:** $n.\text{toString}(2) \rightarrow 32 \text{ digits కి pad} \rightarrow \text{string reverse} \rightarrow \text{parseInt}(\text{rev}, 2)$. పని చేస్తుంది కానీ string manipulation నెమ్మది, padding గజిబిజి. Bit ops తో నేరుగా చేయడం clean & fast.
- **Optimal Approach: Bit-by-bit build.** 32 iterations: $\text{result} = (\text{result} \ll 1) | (n \& 1)$, తర్వాత $n \gg= 1$. చివర్లో **result** $\gg 0$ – ఎందుకంటే $\ll$ signed 32-bit ఇస్తుంది; top bit set అయితే negative అవుతుంది, $\gg 0$ దాన్ని unsigned గా మారుస్తుంది. $O(1)$ (32 స్థిరం).
- **Solution (JavaScript):**

```
/**
 * @param {number} n - 32-bit unsigned integer (అనే అనుకుంటాం)
 * @return {number} - bits reversed, unsigned
 */
var reverseBits = function (n) {
    let result = 0;

    for (let i = 0; i < 32; i++) {
        // result ని ఎడమకి జరిపి, n యొక్క lowest bit ని కుడివైపు పెట్టు
        result = (result << 1) | (n & 1);
        // n ని కుడికి జరుపు – UNSIGNED shift (sign propagate కాకూడదు)
        n >>= 1;
    }

    // << signed 32-bit ఇస్తుంది → top bit set అయితే negative.
    // >> 0 తో unsigned 32-bit గా reinterpret చెయ్యి.
    return result >> 0;
};
```

- **Dry Run (చిన్న 4-bit ఉదాహరణతో idea):** $n = 1011$ (4 bits అనుకో). **result** 0 తో మొదలు.

step	n (binary)	n&1	result before	result = (result<<1) (n&1)	n>>=1
1	1011	1	0000	0001	101
2	0101	1	0001	0011	010
3	0010	0	0011	0110	001
4	0001	1	0110	1101	000

$1011 \rightarrow 1101$ – సరిగ్గా reversed.  (నిజ problem లో ఇది 32 సార్లు జరుగుతుంది.)

- **Complexity:** Time $O(1)$ – ఎప్పుడూ సరిగ్గా 32 iterations. Space $O(1)$.
- **గుర్తుంచుకోవాల్సినది:** "lowest bit తీసి highest లో పెట్టు, 32 సార్లు" = reversal. ముఖ్య takeaway – JS లో bit shifting లో $\gg$ (**unsigned**) వాడాలి, ముఖ్యంగా input unsigned అయితే, మరియు final answer కి $\gg 0$. ఇది

"reverse", "swap adjacent bits", "rotate bits" లాంటి అన్ని bit-layout problems కీ పునాది.

- **సాధారణ తప్పులు:**

- $n \gg= 1$ (signed) వాడటం $n \gg 1$ బదులు $\rightarrow$ top bit set అయిన inputs కి infinite loop / తప్పు answer (sign bit మళ్ళీ మళ్ళీ వస్తుంది).
- చివర్లో $\gg 0$ మర్చిపోవడం $\rightarrow$ top bit set అయితే negative number return అవుతుంది (LeetCode fail).
- 32 బదులు తక్కువ iterations (leading zeros ని మర్చిపోవడం) – ఎప్పుడూ పూర్తి 32.

5. Number of 1 Bits (LeetCode #191) – Easy

- **సమస్య:** ఒక integer n ఇస్తారు. దాని binary representation లో ఎన్ని **1 bits (set bits)** ఉన్నాయో లెక్కించి return చెయ్యాలి. దీన్ని **Hamming weight** అంటారు.
- **Key constraints:** Input 32-bit range. (పాత version లో input ఒక binary string; కొత్త version లో plain integer – logic ఒకటే.)
- **ఉదాహరణ:**

```
Input:  n = 11    (binary: 1011)
Output: 3
వివరణ: 1011 లో మూడు 1లు ఉన్నాయి.
```

- **ఎలా ఆలోచించాలి:**

సులభమైన ఆలోచన: 32 bits మీద నడిచి, ప్రతి bit $(n \gg i) \& 1$ చూసి, 1 అయితే count. ఇది పని చేస్తుంది, ఎప్పుడూ 32 steps.

కానీ ఇంకా చక్కని trick ఉంది (Brian Kernighan): $n \& (n-1)$ operation **lowest set bit ని ఒకేసారి తీసివేస్తుంది** (bit primer లో చూశాం). కాబట్టి n ను 0 అయ్యేదాకా $n = n \& (n-1)$ చేస్తూ ఎన్నిసార్లు చేశామో లెక్కిస్తే – అదే set bits count! తేడా ఏంటంటే – ఇది bits **ఎన్ని set ఉంటే అన్నిసార్లు** loop తిరుగుతుంది (32 కాదు). Sparse numbers కి చాలా fast.

$n-1$ ఎందుకు lowest 1 ని తీస్తుంది? $n-1$ అనేది lowest set bit ని 0 చేసి, దాని కుడివైపు అన్ని 0లను 1లు చేస్తుంది. ఆ తర్వాత $\&$ వేస్తే ఆ lowest bit + కుడివైపువన్నీ 0 అవుతాయి, మిగతా bits అలాగే ఉంటాయి $\rightarrow$ సరిగ్గా ఒక్క set bit పోతుంది.

- **Brute Force / naive:** 32 iterations, ప్రతిదానికీ $(n \gg i) \& 1$ చూసి count. Time $O(32) = O(1)$, పని చేస్తుంది. కానీ set bits తక్కువ ఉన్నా ఎప్పుడూ 32 steps.
- **Optimal Approach: Brian Kernighan's** $n \&= (n-1)$. ప్రతి iteration ఒక set bit ని తీస్తుంది; loop set-bit-count సార్లు తిరుగుతుంది. $O(k)$ where $k = \text{set bits} (\leq 32)$. Space $O(1)$.
- **Solution (JavaScript):**

```
/**
 * @param {number} n
```

```

* @return {number} - set bits (1s) count
*/
var hammingWeight = function (n) {
    let count = 0;

    while (n !== 0) {
        n &= (n - 1); // lowest set bit ని తీసేయి (ఒక 1 పోతుంది)
        count++;      // తీసిన ప్రతిసారీ లెక్కించు
    }

    return count;
};

```

ప్రత్యామ్నాయం (32-bit scan, >>> వాడుతూ) – top bit set అయిన inputs కి safe:

```

var hammingWeight = function (n) {
    let count = 0;
    for (let i = 0; i < 32; i++) {
        count += (n >>> i) & 1; // ప్రతి bit ని unsigned గా చదువు
    }
    return count;
};

```

- **Dry Run:** $n = 11 = 1011$.

step	n (binary)	n-1 (binary)	n & (n-1)	count
1	1011	1010	1010 (=10)	1
2	1010	1001	1000 (=8)	2
3	1000	0111	0000 (=0)	3

$n = 0 \rightarrow$ ఆగు. $\text{count} = 3$.  ప్రతిసారీ కుడివైపు మొదటి 1 పోవడం గమనించు.

- **Complexity:** Time $O(k)$ (k = set bits, ≤ 32) $\rightarrow$ practically $O(1)$. Space $O(1)$.
- **గుర్తుంచుకోవాల్సినది:** $n \& (n-1)$ = "lowest set bit ని తీసేయి" – ఇది అత్యంత శక్తివంతమైన bit trick. Set bits లెక్కించడం, "power of 2 నా?" ($n > 0$ && $(n \& (n-1)) === 0$), "ఒక్క bit మార్పుతో equal అవుతాయా" – అన్నిటికీ ఇదే. Kernighan trick set-bit-count సార్లే తిరుగుతుందని గుర్తుంచుకో.
- **సాధారణ తప్పులు:**
 - Signed shift $>>$ వాడి, top-bit-set inputs (large numbers) కి infinite loop లేదా over-count. Scan version లో ఎప్పుడూ $>>>$.
 - $n \& (n-1)$ ని $n \& (n+1)$ గా రాయడం (అది highest బదులు వేరే పని).
 - $\text{while} (n > 0)$ వాడటం $\text{while} (n !== 0)$ బదులు – top bit set అయితే n negative గా చదవబడి loop skip అవుతుంది. $!== 0$ safe.

6. Single Number (LeetCode #136) — Easy

- **సమస్య:** ఒక integer array `nums` ఇస్తారు. అందులో **ప్రతి element సరిగ్గా రెండుసార్లు** వస్తుంది — **ఒక్కటి తప్ప** (అది ఒకేసారి). ఆ ఒంటరి (single) element ని కనుక్కోవాలి.
- **Key constraints:** Linear time $O(n)$ మరియు constant $O(1)$ extra space లో చెయ్యాలి (interview requirement — hashmap వాడకూడదు).
- **ఉదాహరణ:**

```
Input:  nums = [4,1,2,1,2]
Output: 4
వివరణ: 1 రెండుసార్లు, 2 రెండుసార్లు, 4 ఒక్కసారి → 4.
```

- **ఎలా ఆలోచించాలి:**

మొదటి ఆలోచన: hashmap లో counts పెట్టి, count == 1 ఉన్నది వెతకడం. కానీ అది $O(n)$ space — constraint ఉల్లంఘన.

ఏం గమనించాలి? "ప్రతిదీ జతగా, ఒక్కటి తప్ప" — ఈ మాట వినగానే **XOR** గుర్తు రావాలి! XOR యొక్క రెండు జాదూ లక్షణాలు: $a \oplus a = 0$ (ఒకే number రెండుసార్లు XOR చేస్తే cancel), $a \oplus 0 = a$. కాబట్టి అన్ని numbers ని XOR చేస్తే — జతలుగా వచ్చినవన్నీ 0 గా cancel అయిపోతాయి, మిగిలేది **ఒంటరి number మాత్రమే**. Order కూడా అనవసరం (XOR commutative).

- **Brute Force / naive:** Hashmap/Set తో counts. Time $O(n)$, Space $O(n)$. లేదా nested loop $O(n^2)$ time, $O(1)$ space. రెండూ constraint ($O(n)$ time + $O(1)$ space కలిపి) ని satisfy చెయ్యవు.
- **Optimal Approach: అన్నీ XOR చెయ్యి.** జతలు cancel → ఒంటరిది మిగులుతుంది. $O(n)$ time, $O(1)$ space — perfect.
- **Solution (JavaScript):**

```
/**
 * @param {number[]} nums
 * @return {number} - ఒంటరిగా (ఒకేసారి) వచ్చే element
 */
var singleNumber = function (nums) {
  let result = 0; // 0 తో మొదలు (a ^ 0 = a కాబట్టి safe)

  for (const num of nums) {
    result ^= num; // జతలు cancel అవుతాయి, ఒంటరిది మిగులుతుంది
  }

  return result;
};
```

- **Dry Run:** `nums = [4,1,2,1,2]`. `result` 0 తో మొదలు (binary చూపిస్తూ):

step	num	result before	result ^ num	result (binary)
1	4	000	0^4	100 (4)
2	1	100	4^1	101 (5)
3	2	101	5^2	111 (7)
4	1	111	7^1	110 (6)
5	2	110	6^2	100 (4)

result = 4. ✅ గమనించు: 1 రెండుసార్లు, 2 రెండుసార్లు వచ్చి cancel అయ్యాయి; మిగిలింది 4.

- **Complexity:** Time $O(n)$ — ఒక్క pass. Space $O(1)$ — ఒక్క variable, hashmap లేదు.
- **గుర్తుంచుకోవాల్సినది: "జతలు cancel అవ్వాలంటే XOR."** $a \oplus a = 0$, $a \oplus 0 = a$ — ఈ రెండు లక్షణాలు గుర్తుంచుకుంటే, "duplicate తప్ప అన్నీ జతలు", "missing number", "రెండు single numbers" (Single Number III) లాంటివి అన్నీ XOR తో $O(1)$ space లో పరిష్కారం. XOR = "space లేకుండా జ్ఞాపకం."
- **సాధారణ తప్పులు:**
 - `result` ని 0 బదులు `nums[0]` తో initialize చేసి, తర్వాత మళ్ళీ `nums[0]` ను XOR చెయ్యడం (double counting). 0 తో మొదలుపెట్టి అన్నిటిని XOR చెయ్యి.
 - Hashmap వాడేసి $O(n)$ space constraint ను ఉల్లంఘించడం (interview లో reject).
 - "ప్రతిదీ మూడుసార్లు, ఒక్కటి తప్ప" అనే variant (త్వర్కాతి problem) కి plain XOR సరిపోదు అని గుర్తించకపోవడం — అక్కడ వేరే technique.

7. Single Number II (LeetCode #137) — Medium

- **సమస్య:** ఒక integer array `nums` — అందులో ప్రతి element సరిగ్గా మూడుసార్లు వస్తుంది, ఒక్కటి తప్ప (అది ఒకేసారి). ఆ ఒంటరి element ని కనుక్కోవాలి.
- **Key constraints:** $O(n)$ time, $O(1)$ extra space. Numbers **negative** కూడా కావచ్చు ($-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$) — sign handling ముఖ్యం.
- **ఉదాహరణ:**

Input: `nums = [2,2,3,2]`
Output: 3
వివరణ: 2 మూడుసార్లు, 3 ఒకసారి → 3.

- **ఎలా ఆలోచించాలి:**

ఇక్కడ plain XOR పని చెయ్యదు — ఎందుకంటే ఒక number మూడుసార్లు XOR చేస్తే cancel కాదు ($a \oplus a \oplus a = a$, 0 కాదు). "జతలు" కాదు, "మూడు" — కొత్త idea కావాలి.

Bit-counting insight: ప్రతి bit position (0 నుండి 31 దాకా) ని విడిగా చూద్దాం. ప్రతి number మూడుసార్లు వస్తుంది కాబట్టి — ఏ bit position లోనైనా, triple-వచ్చే numbers ఆ bit కి 3 యొక్క గుణిజం (multiple of 3) సార్లు contribute

చేస్తాయి. ఒంటరి number మాత్రం ఆ bit ని **0 లేదా 1 సారి** అదనంగా contribute చేస్తుంది. కాబట్టి **ప్రతి bit position లో 1ల మొత్తాన్ని లెక్కించి % 3 చేస్తే** – మిగిలేది ఒంటరి number యొక్క ఆ bit! ఇలా 32 bits కి చేస్తే ఒంటరి number reconstruct అవుతుంది.

Negatives కి జాగ్రత్త: bit చదవడానికి `>>>` (**unsigned**) వాడాలి – లేకపోతే sign bit propagate అయి counts తప్పువుతాయి.

- **Brute Force / naive:** Hashmap లో counts, count == 1 వెతకడం. $O(n)$ time కానీ $O(n)$ space – constraint violate. Sort చేసి triples skip చేయడం $O(n \log n)$.
- **Optimal Approach (సులభం, గుర్తుంచుకోదగ్గది):** 32 bit positions మీద నడిచి, ప్రతి position లో set bits count % 3 . % 3 != 0 అయిన bits ని answer లో set చెయ్యి. $O(32n) = O(n)$ time, $O(1)$ space. (Advanced $O(1)$ -time-per-element variant కింద ఇచ్చాను.)
- **Solution (JavaScript):**

```
/**
 * @param {number[]} nums
 * @return {number} - మూడుసార్లు కాక ఒకసారి వచ్చే element
 */
var singleNumber = function (nums) {
    let result = 0;

    // ప్రతి bit position (0..31) విడిగా చూడు
    for (let i = 0; i < 32; i++) {
        let bitSum = 0;
        for (const num of nums) {
            bitSum += (num >>> i) & 1; // >>> UNSIGNED - negatives కి కీలకం
        }
        // triples 3 యొక్క గుణిజం contribute చేస్తాయి; మిగిలించి ఒంటరిది
        if (bitSum % 3 !== 0) {
            result |= (1 << i); // ఈ bit ని answer లో set చెయ్యి
        }
    }

    // result ఇప్పటికే 32-bit signed (bit 31 set అయితే negative - అదే correct).
    return result;
};
```

Advanced ప్రత్యామ్నాయం (ones/twos state machine, $O(1)$ space, ఒక్క pass):

```
var singleNumber = function (nums) {
    let ones = 0, twos = 0;
    for (const num of nums) {
        ones = (ones ^ num) & ~twos; // ఒకసారి కనిపించిన bits
        twos = (twos ^ num) & ~ones; // రెండుసార్లు కనిపించిన bits
    }
    return ones; // మూడోసారికి రెండూ clear → ఒంటరిది ones లో మిగులుతుంది
};
```

- **Dry Run (bit-count approach):** `nums = [2,2,3,2]` . `2 = 010` , `3 = 011` .

bit i	numbers లో ఆ bit 1లు (2,2,3,2)	bitSum	% 3	answer bit
0	2→0, 2→0, 3→1, 2→0	1	1	set (1)
1	2→1, 2→1, 3→1, 2→1	4	1	set (2)
2..31	అన్నీ 0	0	0	not set

Answer bits: bit0 + bit1 = 011 = 3 . ✓

- **Complexity:** Time $O(32n) = O(n)$. Space $O(1)$.
- **గుర్తుంచుకోవాల్సినది:** "XOR (జతలకి) పని చెయ్యనప్పుడు – bit positions ని విడిగా లెక్కించి % k చెయ్యి." "k సార్లు తప్పు ఒక్కటి" అనే ఏ generalization క్షణా (% 3 , % 5 ...) ఇదే bit-counting approach పని చేస్తుంది – చాలా versatile. Negatives ఉంటే >>> మర్చిపోకు.
- **సాధారణ తప్పులు:**
 - Signed >> వాడి negative numbers కి తప్పు bit counts (sign bit fills). ఎప్పుడూ >>> i .
 - 1 << 31 positive 2^{31} అని అనుకోవడం – అది JS లో negative. కానీ ఇక్కడ result signed 32-bit గా సరిగ్గానే వస్తుంది (bit 31 set = negative answer, ఇది correct).
 - Plain XOR (Problem 6 solution) ను ఇక్కడ apply చెయ్యడం – triples cancel అవ్వవు, తప్పు answer.

8. Bitwise AND of Numbers Range (LeetCode #201) – Medium

- **సమస్య:** రెండు integers `left` , `right` ఇస్తారు (`left <= right`). `left` నుండి `right` వరకు (రెండూ కలిపి) అన్ని numbers ని bitwise AND చేస్తే వచ్చే result ని return చెయ్యాలి: `left & (left+1) & ... & right` .
- **Key constraints:** $0 \leq left \leq right \leq 2^{31} - 1$. Range చాలా పెద్దదవ్వచ్చు – ఒక్కొక్కటిగా AND చేస్తే TLE.
- **ఉదాహరణ:**

Input: `left = 5, right = 7`
Output: 4
వివరణ: `5 & 6 & 7 = 101 & 110 & 111 = 100 = 4`

- **ఎలా ఆలోచించాలి:**

Range లో అన్నీ AND చేస్తే – ఒక bit position లో ఒక్క number అయినా 0 ఉంటే, ఆ bit final result లో 0 (AND యొక్క స్వభావం). కాబట్టి final result లో 1 గా మిగిలే bits = range అంతటా అన్ని numbers లోనూ 1 ఉన్న bits మాత్రమే.

కీలక observation: Range `[left, right]` లో numbers పెరుగుతున్నప్పుడు, lower bits అన్నీ 0↔1 మారుతూ ఉంటాయి (కనీసం ఒకసారి 0 అవుతాయి) → అవన్నీ final AND లో 0. మిగిలేది – `left` మరియు `right` యొక్క common leading prefix (ఎడమవైపు ఒకేలా ఉన్న bits) మాత్రమే! ఉదా. `5=101` , `7=111` → common prefix `1__` → `100 = 4` .

ప్లాన్: `left` ను `right` కు సమానం అయ్యేదాకా రెండింటినీ కుడికి shift చేస్తూ (`>= 1`), ఎన్ని shifts చేశామో లెక్కించు. చివర్లో common prefix ను తిరిగి ఎడమకి అన్ని shifts జరిపి zeros నింపు.

- **Brute Force / naive:** `for (let i = left; i <= right; i++) result &= i`. Range 10^9 పెద్దదైతే billions of iterations → **TLE**. అంతేకాక result త్వరగా 0 అవ్వచ్చు కానీ దాన్ని ముందే గుర్తించలేం (simple గా).
- **Optimal Approach: Common prefix ను కనుక్కో.** `left`, `right` సమానం అయ్యేదాకా రెండింటినీ `>> 1` చేస్తూ shift count పెంచు; తర్వాత `left << shift`. `0(log(max))` (≤ 32 shifts). Space `O(1)`. (Brian Kernighan variant కూడా ఉంది – కింద.)
- **Solution (JavaScript):**

```
/**
 * @param {number} left
 * @param {number} right
 * @return {number} - [left, right] range అంతా bitwise AND
 */
var rangeBitwiseAnd = function (left, right) {
    let shift = 0;

    // left, right సమానం అయ్యేదాకా common prefix కనుక్కో
    while (left < right) {
        left >>= 1; // రెండింటి లోని differing lower bits ని తొలగిస్తూ
        right >>= 1;
        shift++; // ఎన్ని bits తొలగించామో గుర్తుంచుకో
    }

    // common prefix ను తిరిగి స్థానంలో పెట్టు, కుడివైపు zeros నింపుతూ
    return left << shift;
};
```

ప్రత్యామ్నాయం (Brian Kernighan — `right` యొక్క lowest set bits తొలగిస్తూ):

```
var rangeBitwiseAnd = function (left, right) {
    while (right > left) {
        right &= (right - 1); // right యొక్క lowest set bit తీసేస్తూ
    }
    return right;
};
```

- **Dry Run:** `left = 5 (101)`, `right = 7 (111)`.

step	left (binary)	right (binary)	left < right?	shift
start	101 (5)	111 (7)	yes	0
1	10 (2)	11 (3)	yes	1
2	1 (1)	1 (1)	no → stop	2

Common prefix `left = 1`, `shift = 2` → `1 << 2 = 100 = 4`. ✓

- **Complexity:** Time `O(log(max))` ≤ 32 shifts (`O(1)` practically). Space `O(1)`.

- **గుర్తుంచుకోవాల్సినది: "Range AND = left & right యొక్క common binary prefix, మిగతా bits zeros."** Range మీద ప్రతిదీ iterate చెయ్యాలన్న అవసరం లేదు — bits ఎలా మారతాయో ఆలోచిస్తే shortcut దొరుకుతుంది. AND అంటే "ఒక్క 0 చాలు bit ని చంపడానికి" — ఈ intuition ముఖ్యం.
- **సాధారణ తప్పులు:**
 - Naive loop `left` నుండి `right` దాకా — పెద్ద ranges కి TLE.
 - `shift` తిరిగి apply చెయ్యడం మర్చిపోవడం — common prefix ను సరైన స్థానంలో పెట్టాలి (`left << shift`).
 - `left == right` (single number) case — loop నడవదు, `shift = 0` , `left << 0 = left` correct గా return అవుతుంది (edge handled).
 - `left = 0` అయితే — result ఎప్పుడూ 0 (0 తో AND). Loop సహజంగా దీన్ని handle చేస్తుంది (`0 < right` అయితే `shift` అవుతూ 0 వస్తుంది).

Pattern: Math Tricks

వివరణ

Math problems లో fancy data structures అవసరం లేదు — కావాల్సింది కొన్ని **elementary number properties** ను గుర్తుంచుకోవడం. LeetCode math problems దాదాపు ఎప్పుడూ ఈ నాలుగు tools మీదే ఆధారపడతాయి:

1. Digits ని విడదీయడం (`% 10` మరియు `/ 10`):

ఏ number నుండినా చివరి digit తీయాలంటే `n % 10` , ఆ digit ను తొలగించాలంటే `Math.floor(n / 10)` .

```
n = 1234
n % 10      = 4      (చివరి digit)
Math.floor(n / 10) = 123 (చివరి digit తొలగించాక)
```

ఇలా loop లో అన్ని digits ను కుడి నుండి ఎడమకి తీయవచ్చు. Number ను reverse చెయ్యడం: `rev = rev * 10 + n % 10` .

2. GCD (Greatest Common Divisor) — Euclid's algorithm:

రెండు numbers యొక్క అతిపెద్ద ఉమ్మడి భాజకం. `gcd(a, b) = gcd(b, a % b)` , `b = 0` అయ్యేదాకా. Fractions ను simplify చెయ్యడానికి (ఉదా. slope `dy/dx`) చాలా అవసరం.

3. Fast Exponentiation (binary exponentiation) — `x^n` ను `O(log n)` లో:

`x^n` ను `n` సార్లు గుణించడం `O(n)` — నెమ్మది. బదులుగా, exponent ను binary గా చూడు: `x^n` లో, `n` యొక్క ప్రతి set bit కి, base యొక్క వరుస **squares** (`x`, `x^2`, `x^4`, `x^8`...) ను గుణిస్తాం. `n` ప్రతిసారీ సగం అవుతుంది $\rightarrow O(\log n)$.

4. Factor counting (ఒక number ఎన్నిసార్లు భాగిస్తుందో):

`n!` లో ఒక prime `p` ఎన్నిసార్లు factor గా వస్తుందో $= \lfloor n/p \rfloor + \lfloor n/p^2 \rfloor + \lfloor n/p^3 \rfloor + \dots$ (Legendre's formula). Trailing zeros లాంటి problems కి కీలకం.

Real-life analogy: Fast exponentiation ను ఇలా అనుకో – నీకు x ను 16 సార్లు గుణించాలి. ఒక్కొక్కటిగా చేస్తే 16 గుణింపులు. కానీ నువ్వు తెలివిగా: $x \cdot x = x^2$, $x^2 \cdot x^2 = x^4$, $x^4 \cdot x^4 = x^8$, $x^8 \cdot x^8 = x^{16}$ – కేవలం 4 గుణింపులు! ప్రతిసారీ ఉన్నదాన్ని రెట్టింపు చేస్తూ పోవడం. ఇదే "repeated squaring" – logarithmic magic. (బట్టీ కొట్టడం vs concepts ను రెట్టింపు చేసుకుంటూ నేర్చుకోవడం లాంటిది.)

ఎలా గుర్తించాలి (recognition signals)

- "digits", "reverse a number", "palindrome number", "sum of digits" → `% 10`, `/ 10`.
- "x raised to n", "power", పెద్ద exponents → fast exponentiation.
- "trailing zeros", "count factors", "divisible by" → factor counting.
- "slope", "fraction", "ratio", "simplify" → gcd.
- "sqrt", "find X such that $X^2 \leq n$ " → binary search on answer (math + search).

Reusable Templates

```
// 1. Digits iterate (కుడి నుండి ఎడమకి)
while (n > 0) {
    const digit = n % 10;
    n = Math.floor(n / 10);
}

// 2. GCD (Euclid)
function gcd(a, b) {
    return b === 0 ? a : gcd(b, a % b);
}

// 3. Fast exponentiation ( $x^n$ ,  $n \geq 0$ )
function power(x, n) {
    let result = 1;
    while (n > 0) {
        if (n % 2 === 1) result *= x; // odd bit → base ను గుణించు
        x *= x;                       // base ను square చెయ్యి
        n = Math.floor(n / 2);        // exponent ను సగం చెయ్యి
    }
    return result;
}
```

Complexity

- Digit ops: $O(d)$ where d = digits count = $O(\log_{10} n)$.
- GCD: $O(\log(\min(a, b)))$.
- Fast exponentiation: $O(\log n)$ – n సార్లు కాక $\log n$ సార్లు.

9. Palindrome Number (LeetCode #9) — Easy

- సమస్య:** ఒక integer x ఇస్తారు. అది **palindrome** (ముందు నుండి, వెనుక నుండి ఒకేలా చదవబడేది) అయితే `true`, కాకపోతే `false` return చెయ్యాలి. Follow-up: string గా convert చెయ్యకుండా చేయగలవా?
- Key constraints:** $-2^{31} \leq x \leq 2^{31} - 1$. Negative numbers ఎప్పుడూ palindrome కాదు (`-121` reversed `121-` — ❌ వల్ల కాదు).
- ఉదాహరణ:**

```
Input:  x = 121      Output: true
Input:  x = -121     Output: false (negative)
Input:  x = 10       Output: false (01 ≠ 10)
```

- ఎలా ఆలోచించాలి:**

సులభమైనది: `x.toString()` ను reverse చేసి పోల్చడం. కానీ follow-up "string వాడకుండా" అంటే — number ను **గణితంగా reverse** చెయ్యాలి (`% 10`, `/ 10` తో), తర్వాత original తో పోల్చు.

ఒక తెలివైన optimization: మొత్తం number ను reverse చేస్తే overflow రావొచ్చు. బదులుగా **సగం digits మాత్రమే reverse** చేద్దాం — reversed half, మిగిలిన half కి సమానం అయ్యేదాకా. `revertedHalf >= remainingX` అయినప్పుడు మనం సగం దాటాం. Even length అయితే `x === reverted`; odd length అయితే మధ్య digit ను వదిలేసి `x === Math.floor(reverted/10)`.

Early exits: `x < 0` → false. చివర్లో 0 ఉన్న numbers (`x % 10 === 0`) 0 తప్ప palindrome కావు (leading zero ఉండదు కాబట్టి, ఉదా. `10` → `01` invalid).

- Brute Force / naive:** String convert + two-pointer/reverse compare. Time $O(d)$, Space $O(d)$ (string). పని చేస్తుంది కానీ follow-up ను satisfy చెయ్యదు.
- Optimal Approach:** సగం number ను గణితంగా reverse. $O(d)$ time, $O(1)$ space, string లేదు, overflow లేదు.
- Solution (JavaScript):**

```
/**
 * @param {number} x
 * @return {boolean}
 */
var isPalindrome = function (x) {
  // negative కాదు; 0 తో ముగిసేవి (0 తప్ప) palindrome కావు
  if (x < 0 || (x % 10 === 0 && x !== 0)) return false;

  let reverted = 0;
  // సగం digits reverse అయ్యేదాకా (reverted, x ను దాటేదాకా)
  while (x > reverted) {
    reverted = reverted * 10 + (x % 10); // చివరి digit ను reverted కి జోడించు
    x = Math.floor(x / 10);           // x నుండి ఆ digit తొలగించు
  }
}
```

```
// even length: x === reverted
// odd length: మధ్య digit reverted లో ఉంటుంది → దాన్ని వదిలి పోల్చు
return x === reverted || x === Math.floor(reverted / 10);
};
```

- **Dry Run:** $x = 1221$.

step	x (before)	x % 10	reverted = rev*10 + digit	x = floor(x/10)	x > reverted?
1	1221	1	$0*10+1 = 1$	122	$122 > 1$ yes
2	122	2	$1*10+2 = 12$	12	$12 > 12$ no → stop

$x = 12$, reverted = 12 . $x === reverted \rightarrow \text{true}$. ✓ (121 కి: stop వద్ద $x=1$, reverted=12 ; $1 === \text{floor}(12/10)=1 \rightarrow \text{true}$.)

- **Complexity:** Time $O(d) = O(\log_{10} x)$ — సగం digits. Space $O(1)$.
- **గుర్తుంచుకోవాల్సినది:** "Number ను string చెయ్యకుండా digits తో ఆడాలంటే $\% 10$, $/ 10$." సగం మాత్రమే reverse చేసే trick overflow ను తప్పిస్తుంది — "మొత్తం అవసరం లేకపోతే సగంతోనే ఆపు" అనే ఆలోచన చాలా చీట్లు ఉపయోగం. Negative & trailing-zero edge cases ముందే handle చెయ్యి.
- **సాధారణ తప్పులు:**
 - Negative numbers ను palindrome అనుకోవడం (-121 false).
 - 10 , 100 లాంటి trailing-zero numbers ను miss చెయ్యడం — $x \% 10 === 0 \ \&\& \ x !== 0$ guard.
 - Odd-length numbers కి $\text{Math.floor}(\text{reverted}/10)$ (మధ్య digit drop) మర్చిపోవడం → 12321 fail.
 - మొత్తం number reverse చేసి overflow (సగం approach దీన్ని తప్పిస్తుంది).

10. Plus One (LeetCode #66) — Easy

- **సమస్య:** ఒక పెద్ద integer ను దాని digits array **digits** గా ఇస్తారు (most-significant digit మొదట, ప్రతి element 0-9). ఆ integer కి **1 కలిపి**, ఫలిత digits array ను return చెయ్యాలి. Leading zeros ఉండవు.
- **Key constraints:** Number చాలా పెద్దదవచ్చు ($1 \leq \text{digits.length} \leq 100$) — కాబట్టి integer గా convert చేస్తే overflow. Digits array మీదే పని చెయ్యాలి.
- **ఉదాహరణ:**

```
Input: digits = [1,2,3]      Output: [1,2,4]      (123 + 1 = 124)
Input: digits = [9,9,9]      Output: [1,0,0,0]     (999 + 1 = 1000)
```

- **ఎలా ఆలోచించాలి:**

School math: చివరి digit కి 1 కలుపు. అది 9 కాకపోతే — ఒక్కటి పెంచి, అయిపోయింది (carry లేదు). అది 9 అయితే — 0 అవుతుంది, carry ఎడమకి వెళ్తుంది; తర్వాతి digit కి అదే logic.

ఏం గమనించాలి? చివర నుండి నడుస్తూ, $\text{digit} < 9$ కనిపించిన **మొదటి క్షణమే** – దాన్ని పెంచి array return చేయవచ్చు (carry అగిపోతుంది). Loop మొత్తం పూర్తయినా return కాకపోతే – అంటే **అన్నీ 9లు** ([9,9,9]) – అప్పుడు అన్నీ 0 అయ్యాయి, ముందు కొత్త 1 చేర్చాలి ([1,0,0,0]).

- **Brute Force / naive:** `BigInt(digits.join('')) + 1n` తర్వాత తిరిగి digits గా విడదీయడం. పని చేస్తుంది (BigInt overflow తప్పిస్తుంది) కానీ interviewer manual carry logic చూడాలనుకుంటాడు; parse/format overhead అనవసరం.
- **Optimal Approach: చివర నుండి carry.** $\text{digit} < 9$ కనిపిస్తే పెంచి వెంటనే return; లేకపోతే 0 చేసి కొనసాగించు. Loop దాటితే అన్నీ 9లు $\rightarrow$ [1, ...zeros] . $O(n)$ time.
- **Solution (JavaScript):**

```
/**
 * @param {number[]} digits - MSB first, each 0-9
 * @return {number[]}
 */
var plusOne = function (digits) {
  // చివరి digit నుండి ఎడమకి నడుపు
  for (let i = digits.length - 1; i >= 0; i--) {
    if (digits[i] < 9) {
      digits[i]++; // carry లేదు - ఒక్కటి పెంచి పని ముగిసింది
      return digits;
    }
    digits[i] = 0; // 9  $\rightarrow$  0, carry ఎడమకి కొనసాగుతుంది
  }

  // ఇక్కడికి వస్తే అన్నీ 9లు అయ్యాయి (అన్నీ 0 అయ్యాయి)  $\rightarrow$  ముందు 1 చేర్చు
  return [1, ...digits]; // ఉదా. [0,0,0]  $\rightarrow$  [1,0,0,0]
};
```

- **Dry Run:** `digits = [9,9,9]` .

i	digits[i]	< 9?	action	digits
2	9	no	set 0, carry	[9,9,0]
1	9	no	set 0, carry	[9,0,0]
0	9	no	set 0, carry	[0,0,0]

Loop ముగిసింది, return కాలేదు $\rightarrow$ అన్నీ 9లు $\rightarrow$ [1, 0, 0, 0] .  ([1,2,3] కి: $i=2$, $3 < 9 \rightarrow \text{digits}[2]=4$, వెంటనే [1,2,4] return.)

- **Complexity:** Time $O(n)$ – worst case (అన్నీ 9లు) అన్ని digits touch. Space $O(1)$ extra (అన్నీ-9 case లో మాత్రం కొత్త array – output కి తప్పనిసరి).
- **గుర్తుంచుకోవాల్సినది:** "చివర నుండి carry; carry అగిన క్షణమే return." ఇది Add Binary (Problem 3) కి జంట – base 10 అంతే తేడా. Edge case "అన్నీ 9లు $\rightarrow$ పొడవు 1 పెరుగుతుంది" ను మర్చిపోవద్దు; ఇదే ఈ problem యొక్క అసలు test.

- సాధారణ తప్పులు:

- "అన్నీ 9లు" case ([9,9] → [1,0,0]) ను handle చెయ్యకపోవడం – కొత్త leading 1 చేర్చడం మర్చిపోవడం.
- Integer గా convert చేసి పెద్ద inputs కి overflow (digits.length 100 దాకా).
- digits[i] < 9 బదులు <= 9 రాసి, 9ను తప్పుగా పెంచడం (10 అయిపోతుంది).
- చివర [1].concat(digits) బదులు digits.unshift(1) – unshift 0(n) కానీ correct; spread [1, ...digits] clean.

11. Factorial Trailing Zeroes (LeetCode #172) – Medium

- **సమస్య:** ఒక integer n ఇస్తారు. n! (n factorial = 1 × 2 × 3 × ... × n) చివర్లో ఎన్ని **trailing zeros** (వరుస సున్నాలు) ఉంటాయో లెక్కించి return చెయ్యాలి.
- **Key constraints:** 0 ≤ n ≤ 10⁴. n! గణించడం అసాధ్యం (అది భారీ number, overflow) – గణితంగా ఆలోచించాలి. O(log n) కావాలి.
- **ఉదాహరణ:**

Input: n = 5 Output: 1
 వివరణ: 5! = 120 → చివర్లో ఒక్క 0.
 Input: n = 10 Output: 2 (10! = 3628800 → రెండు 0లు)

- ఎలా ఆలోచించాలి:

n! ను నిజంగా లెక్కించడం అసాధ్యం (అతి పెద్దది). గణితం వైపు తిరుగుదాం.

ఒక trailing zero ఎప్పుడు వస్తుంది? ఒక factor 10 వచ్చినప్పుడు. 10 = 2 × 5. కాబట్టి trailing zeros count = n! లో ఎన్ని (2 × 5) జతలు తయారవుతాయో.

కీలక observation: n! లో 2లు, 5ల కంటే చాలా ఎక్కువ (ప్రతి రెండో number 2 ను ఇస్తుంది, కానీ ప్రతి అయిదో number మాత్రమే 5 ను). కాబట్టి **5ల సంఖ్య bottleneck** – trailing zeros = n! లో factor 5 ఎన్నిసార్లు వస్తుందో.

1..n లో 5 యొక్క గుణిజాలు: [n/5]. కానీ 25 = 5×5 రెండు 5లు ఇస్తుంది, 125 మూడు... కాబట్టి [n/5] + [n/25] + [n/125] + ... (Legendre's formula). ప్రతిసారి n ను 5తో భాగిస్తూ, quotient లను కూడితే చాలు.

- **Brute Force / naive:** n! ను (BigInt తో) లెక్కించి, % 10 === 0 అయినంతసేపు digits లెక్కించడం. n = 10⁴ కి n! కి వేల digits – గణన నెమ్మది, memory heavy. అనవసరం.
- **Optimal Approach: 5ల factors ను లెక్కించు:** count = [n/5] + [n/25] + while (n >= 5) { n = [n/5]; count += n; } – ప్రతి step 5 యొక్క తర్వాతి power ను లెక్కిస్తుంది. O(log₅ n) time, O(1) space.
- **Solution (JavaScript):**

```
/**
 * @param {number} n
 * @return {number} - n! లో trailing zeros count
 */
var trailingZeroes = function (n) {
```

```

let count = 0;

// ప్రతి iteration: 5, 25, 125, ... యొక్క గుణిజాలను కూడుతూ
while (n >= 5) {
    n = Math.floor(n / 5); // 5 యొక్క తర్వాతి power స్థాయికి దిగు
    count += n;           // ఆ స్థాయిలో 5ల contribution
}

return count;
};

```

- **Dry Run:** $n = 100$.

step	$n = \text{floor}(n/5)$	count += n	count
1	$100/5 = 20$	+20	20
2	$20/5 = 4$	+4	24
3	$4/5 = 0$	stop ($n < 5$)	24

$100!$ కి trailing zeros = 24. ✓ ($\lfloor 100/5 \rfloor = 20 + \lfloor 100/25 \rfloor = 4 + \lfloor 100/125 \rfloor = 0 = 24$.)

- **Complexity:** Time $O(\log_5 n)$ — ప్రతిసారి n 5 రెట్లు తగ్గుతుంది. Space $O(1)$.
- **గుర్తుంచుకోవాల్సినది:** "Trailing zeros = factor 10ల సంఖ్య = $\min(2\text{ల సంఖ్య}, 5\text{ల సంఖ్య}) = 5\text{ల సంఖ్య}$." పెద్ద products/factorials గురించి అడిగినప్పుడు — నేరుగా లెక్కించకుండా **prime factorization** వైపు ఆలోచించు. "ఏ prime bottleneck?" అనే ప్రశ్న ఈ తరహా problems కి తాళం చెవి. Legendre's formula ($\lfloor n/p \rfloor + \lfloor n/p^2 \rfloor + \dots$) గుర్తుంచుకో.
- **సాధారణ తప్పులు:**
 - కేవలం $\lfloor n/5 \rfloor$ return చెయ్యడం — 25, 125... (అదనపు 5లు) ను మర్చిపోవడం ($n=25$ కి answer 6, 5 కాదు).
 - $n!$ ను నిజంగా లెక్కించడానికి ప్రయత్నించి overflow/timeout.
 - $d = 5$; $d *= 5$ రూపంలో రాస్తే పెద్ద n కి d overflow అవ్వచ్చు; $n = \lfloor n/5 \rfloor$ రూపం safe (n తగ్గుతుంది, పెరగదు).
 - 2లను కూడా లెక్కించి min తీసుకోవడం — అనవసరం (5లే ఎప్పుడూ తక్కువ).

12. Sqrt(x) (LeetCode #69) — Easy

- **సమస్య:** ఒక non-negative integer x ఇస్తారు. దాని **square root ను floor చేసిన (పూర్ణాంక భాగం)** value ను return చెయ్యాలి. అంటే $\sqrt{x}$ ను round-down చేసిన integer. Built-in `Math.sqrt` వాడకూడదు (అదే ఉద్దేశ్యం).
- **Key constraints:** $0 \leq x \leq 2^{31} - 1$. Result integer (దశాంశ భాగం వదిలేయి). ఉదా. $\sqrt{8} \approx 2.82 \rightarrow 2$.
- **ఉదాహరణ:**

Input: x = 8 Output: 2 ($\sqrt{8} = 2.828\dots$, floor $\rightarrow 2$)
Input: x = 4 Output: 2

- ఎలా ఆలోచించాలి:

మనం వెతికేది: **అతిపెద్ద integer r అలా $r^2 \leq x$** . r విలువ $[0, x]$ లో monotonic — r పెరిగితే r^2 పెరుగుతుంది. Monotonic + "అతిపెద్ద/అతిచిన్న X such that condition" $\Rightarrow$ **binary search on the answer**.

$[1, x/2]$ ($x \geq 2$ కి $\sqrt{x} \leq x/2$) లో binary search. $mid^2 \leq x$ అయితే mid ఒక candidate, ఇంకా పెద్దది try చేద్దాం ($lo = mid+1$); లేకపోతే చిన్నది ($hi = mid-1$).

⚠ **Overflow trap:** $mid * mid$ ను నేరుగా లెక్కిస్తే — x పెద్దదైనప్పుడు ($\sim 2 \times 10^9$), $mid \sim 10^9$ అవ్వచ్చు, $mid^2 \sim 10^{18}$ — ఇది `Number.MAX_SAFE_INTEGER` ($2^{53} \approx 9 \times 10^{15}$) ను దాటి **precision కోల్పోతుంది** $\rightarrow$ తప్పు పోలిక! పరిష్కారం: $mid * mid \leq x$ బదులు $mid \leq \text{Math.floor}(x / mid)$ అని division తో పోల్చు (అదే అర్థం, overflow లేదు).

- **Brute Force / naive:** i = 1 నుండి $i*i > x$ అయ్యేదాకా పెంచుతూ వెళ్ళి i-1 return. $O(\sqrt{x})$ — $x = 2^{31}$ కి ~ 46000 steps, పని చేస్తుంది కానీ binary search $O(\log x)$ చాలా వేగం.
- **Optimal Approach: Binary search on answer** in $[1, x/2]$, $mid \leq x/mid$ తో overflow-safe పోలిక. $O(\log x)$ time, $O(1)$ space. (Newton's method కూడా ఉంది కానీ binary search interview కి clean.)
- **Solution (JavaScript):**

```
/**
 * @param {number} x
 * @return {number} - floor(sqrt(x))
 */
var mySqrt = function (x) {
  if (x < 2) return x; // 0  $\rightarrow$  0, 1  $\rightarrow$  1 (edge cases)

  let lo = 1, hi = Math.floor(x / 2), ans = 1;

  while (lo <= hi) {
    const mid = lo + Math.floor((hi - lo) / 2); // overflow-safe midpoint

    // mid*mid పెద్ద x కి  $2^{53}$  దాటి precision పోతుంది  $\rightarrow$  division తో పోల్చు.
    // mid <= floor(x/mid)  $\Leftrightarrow$  mid*mid <= x (positive mid కి)
    if (mid <= Math.floor(x / mid)) {
      ans = mid; // candidate — ఇంకా పెద్దది సాధ్యమో చూద్దాం
      lo = mid + 1;
    } else {
      hi = mid - 1; // mid చాలా పెద్దది
    }
  }

  return ans;
};
```

- **Dry Run:** x = 8. lo=1, hi=4, ans=1.

lo	hi	mid	floor(x/mid)	mid ≤ that?	action	ans
1	4	2	8/2=4	2≤4 yes	lo=3, ans=2	2
3	4	3	8/3=2	3≤2 no	hi=2	2
3	2	—	—	lo>hi stop		2

Return ans = 2. ✓ ($\sqrt{8} = 2.82$, floor 2.)

- **Complexity:** Time $O(\log x)$ — search space ప్రతిసారీ సగం. Space $O(1)$.
- **గుర్తుంచుకోవాల్సినది:** "అతిపెద్ద/అతిచిన్న X such that (monotonic condition) $\Rightarrow$ binary search on the answer." ఇది sqrt మాత్రమే కాదు — "capacity to ship packages", "koko eating bananas", "split array" లాంటి చాలా problems కి పునాది. ఇక్కడ మరో golden lesson: **$a \cdot b$ overflow భయమైతే, $a \leq \text{limit}/b$ గా మార్పు.**
- **సాధారణ తప్పులు:**
 - **$\text{mid} * \text{mid}$** నేరుగా వాడి, పెద్ద x కి precision loss $\rightarrow$ తప్పు answer. Division comparison వాడు.
 - **$\text{mid} = (\text{lo} + \text{hi}) / 2$** లో **Math.floor** మర్చిపోవడం $\rightarrow$ mid దశాంశం అవుతుంది, index logic చెడుతుంది.
 - $x < 2$ (0, 1) edge cases handle చెయ్యకపోవడం ($\text{hi} = x/2 = 0$ అవుతుంది, loop నడవదు, తప్పు 1/0).
 - "ceil" vs "floor" గందరగోళం — problem floor (round down) కోరుతుంది; ans ను candidate వద్ద update చేయడం floor ను ఇస్తుంది.

13. Pow(x, n) (LeetCode #50) — Medium

- **సమస్య:** ఒక double x మరియు integer n ఇస్తారు. x ను n power కి (x^n) లెక్కించి return చెయ్యాలి. n **negative** కావచ్చు (అప్పుడు $x^n = 1 / x^{|n|}$).
- **Key constraints:** $-2^{31} \leq n \leq 2^{31} - 1$. $n \in \text{INT_MIN}(-2^{31})$ కావచ్చు — negation జాగ్రత్త. Built-in **Math.pow** వాడకూడదు.
- **ఉదాహరణ:**

Input: $x = 2.0, n = 10$ Output: 1024.0
Input: $x = 2.0, n = -2$ Output: 0.25 ($1 / 2^2 = 1/4$)

- **ఎలా ఆలోచించాలి:**

Naive: x ను n సార్లు గుణించడం — $O(n)$. $n = 2^{31}$ కి ~2 billion గుణింపులు $\rightarrow$ **TLE**.

Fast exponentiation insight: x^n ను exponent యొక్క binary రూపం ద్వారా వేగంగా. కీలకం: x^n :

- n **even** అయితే $x^n = (x^{\{n/2\}})^2 = (x \cdot x)^{\{n/2\}} \rightarrow$ base ను square చేసి, exponent ను సగం చెయ్యి.
- n **odd** అయితే $x^n = x \cdot x^{\{n-1\}} \rightarrow$ ఒక x ను answer లో పోగుచేసి, మిగతా even.

ఇలా ప్రతిసారీ exponent సగం అవుతుంది $\rightarrow 0(\log n)$. Binary గా చూస్తే: n యొక్క ప్రతి set bit కి, base యొక్క వరుస squares ($x, x^2, x^4 \dots$) ను గుణిస్తున్నాం.

Negative n : $x = 1/x$, $n = -n$ చేసి positive గా మార్చు. **JS లో ఒక ఊరట** — $n = -2^{31}$ ను $-n = 2^{31}$ గా మార్చినా JS numbers (doubles) లో overflow లేదు (C++/Java లో ఇది INT_MIN bug కానీ JS లో కాదు). అయితే exponent 2^{31} దాటుతుంది కాబట్టి bit shift ($n \gg= 1$) బదులు **Math.floor($n/2$)** వాడాలి (bitwise 32-bit లో పరిమితం).

- **Brute Force / naive:** for ($i=0; i<n; i++$) $result *= x$. $O(n)$ — పెద్ద n కి TLE.
- **Optimal Approach: Binary (fast) exponentiation.** n negative అయితే invert. Loop: odd bit అయితే $result *= x$, ప్రతిసారీ $x *= x$, $n = \lfloor n/2 \rfloor$. $O(\log n)$ time, $O(1)$ space.
- **Solution (JavaScript):**

```
/**
 * @param {number} x
 * @param {number} n
 * @return {number} - x^n
 */
var myPow = function (x, n) {
  // negative exponent → reciprocal. (JS doubles: -2^31 negation safe, overflow లేదు)
  if (n < 0) {
    x = 1 / x;
    n = -n;
  }

  let result = 1;
  while (n > 0) {
    // n యొక్క ప్రస్తుత lowest bit 1 అయితే, ప్రస్తుత x (=x^(2^k)) ను పోగుచేయ్యి
    if (n % 2 === 1) result *= x;

    x *= x; // base ను square (తర్వాతి bit కోసం సిద్ధం)
    n = Math.floor(n / 2); // exponent ను సగం (bit shift — కానీ n పెద్దది కావచ్చు, /2 safe)
  }

  return result;
};
```

- **Dry Run:** $x = 2$, $n = 10$ ($10 = \text{binary } 1010$).

n	n%2 (odd?)	result (*x if odd)	x (squared after)	n = $\lfloor n/2 \rfloor$
10	0	1	$2 \rightarrow 4$	5
5	1	$1 \times 4 = 4$	$4 \rightarrow 16$	2
2	0	4	$16 \rightarrow 256$	1
1	1	$4 \times 256 = 1024$	$256 \rightarrow 65536$	0

$n = 0 \rightarrow \text{stop}$. $result = 1024$.  (bits 1 & 3 set: $x^2 \cdot x^8 = 4 \cdot 256 = 1024$.)

- **Complexity:** Time $O(\log n)$ — exponent ప్రతిసారీ సగం. Space $O(1)$ (iterative). Recursive రాష్ట్ర $O(\log n)$ stack.
- **గుర్తుంచుకోవాల్సినది:** "Power/repeated operation ను $O(\log n)$ కి — square the base, halve the exponent." ఈ binary exponentiation matrix power (Fibonacci $O(\log n)$), modular exponentiation (cryptography), "n సార్లు apply" అనే ఏ associative operation క్లైనా వర్తిస్తుంది. "n సార్లు" చూడగానే "log n లో చేయగలనా?" అని ఆలోచించు.
- **సాధారణ తప్పులు:**
 - $n < 0$ handle చెయ్యకపోవడం (reciprocal మర్చిపోవడం).
 - $n \gg 1$ లేదా $n \& 1$ (bitwise) వాడటం — $n = 2^{31}$ bitwise 32-bit range దాటుతుంది; $\text{Math.floor}(n/2)$, $n \% 2$ arithmetic వాడు.
 - $\text{result} *= x$ ను ప్రతి iteration లో (odd check లేకుండా) చెయ్యడం — తప్పు answer.
 - $x *= x$ ను odd-check ముందు చెయ్యడం — square చెయ్యడం result multiply తర్వాత జరగాలి (లేదా క్రమం సరిచూసుకో).

14. Max Points on a Line (LeetCode #149) — Hard

- **సమస్య:** 2D plane లో points array points ($\text{points}[i] = [x_i, y_i]$) ఇస్తారు. **ఒకే సరళరేఖ (straight line) మీద ఉండగలిగే గరిష్ట points సంఖ్య** ను return చెయ్యాలి.
- **Key constraints:** $1 \leq \text{points.length} \leq 300$. అన్ని points **unique** (duplicate points లేవు). Coordinates -10^4 నుండి 10^4 .
- **ఉదాహరణ:**

Input: $\text{points} = [[1,1],[2,2],[3,3]]$ **Output:** 3
వివరణ: మూడూ $y=x$ రేఖ మీద ఉన్నాయి.
Input: $\text{points} = [[1,1],[3,2],[5,3],[4,1],[2,3],[1,4]]$ **Output:** 4

- **ఎలా ఆలోచించాలి:**

రెండు points ఒక రేఖను నిర్వచిస్తాయి. ఒక రేఖ మీద ఉన్న points కి — ఏదో ఒక **anchor** point నుండి చూస్తే, అందరికీ **ఒకే slope** (వాలు) ఉంటుంది. కాబట్టి plan: **ప్రతి point ను anchor గా తీసుకుని**, మిగతా అన్ని points కి slope లెక్కించి, ఒకే slope ఉన్నవి ఎన్నో hashmap లో count చెయ్యి. గరిష్ట $\text{count} + 1$ (anchor ను కలుపుకుని) = ఆ anchor ద్వారా వెళ్ళే రేఖ మీద గరిష్ట points.

⚠ **Slope ను float గా (dy/dx) వాడొద్దు!** Floating-point precision వల్ల $1/3$ లాంటివి తప్పుగా పోల్చబడతాయి, $dx=0$ (vertical line) అయితే division-by-zero. బదులుగా slope ను **reduced fraction ($dy/g, dx/g$)** గా (gcd తో simplify చేసి) string key గా వాడు. Sign ను canonical చెయ్యి (ఉదా. ఎప్పుడూ $dx > 0$) — లేకపోతే $(1,2)$ మరియు $(-1,-2)$ వేరు keys అవుతాయి కానీ అవి ఒకే slope.

- **Brute Force / naive:** ప్రతి మూడు points తీసుకుని collinear నా ($(y_2-y_1)(x_3-x_1) == (y_3-y_1)(x_2-x_1)$) చూడటం — $O(n^3)$. $n=300$ కి 2.7×10^7 , పని చేస్తుంది కానీ slope-map $O(n^2)$ శుభ్రం & వేగం.

- **Optimal Approach: ప్రతి anchor కి slope-count hashmap.** Slope key = gcd-reduced (dx, dy) తో canonical sign. ప్రతి anchor i కి, j > i points మీద slopes count; max(best, count+1) . $O(n^2)$ time, $O(n)$ space.
- **Solution (JavaScript):**

```
/**
 * @param {number[][]} points
 * @return {number}
 */
var maxPoints = function (points) {
    const n = points.length;
    if (n <= 2) return n; // 0,1,2 points ఎప్పుడూ ఒకే రేఖ మీద

    const gcd = (a, b) => (b === 0 ? a : gcd(b, a % b));
    let best = 1;

    for (let i = 0; i < n; i++) {
        const slopeCount = new Map(); // slope-key -> ఎన్ని points

        // i తర్వాతి points మాత్రమే చాలు (ముందువి పాత anchors గా చూశాం)
        for (let j = i + 1; j < n; j++) {
            let dx = points[j][0] - points[i][0];
            let dy = points[j][1] - points[i][1];

            const g = gcd(Math.abs(dx), Math.abs(dy)) || 1; // /0 నివారణ
            dx /= g;
            dy /= g;

            // canonical direction: dx > 0; vertical (dx==0) అయితే dy > 0
            if (dx < 0 || (dx === 0 && dy < 0)) { dx = -dx; dy = -dy; }

            const key = dx + ',' + dy;
            slopeCount.set(key, (slopeCount.get(key) || 0) + 1);
            best = Math.max(best, slopeCount.get(key) + 1); // +1 = anchor ను కలుపు
        }
    }

    return best;
};
```

- **Dry Run:** points = [[1,1],[2,2],[3,3]] . best = 1 .

Anchor i=0 ([1,1]):

- j=1 ([2,2]): dx=1, dy=1, g=1 → key "1,1", count=1, best = max(1, 1+1) = 2 .
- j=2 ([3,3]): dx=2, dy=2, g=2 → dx=1, dy=1, key "1,1", count=2, best = max(2, 2+1) = 3 .

Anchor i=1, i=2 : కొత్త max లేదు. Return best = 3 . ✓ (మూడూ ఒకే slope 1,1 రేఖ మీద.)

- **Complexity:** Time $O(n^2)$ — ప్రతి జత points కి slope (gcd $O(\log)$ కలిపి $O(n^2 \log C)$). Space $O(n)$ — ప్రతి anchor కి slope map.

- గుర్తుంచుకోవాల్సినది: "Collinearity/geometry = ఒక anchor fix చేసి, slope లను gcd-reduced integer fraction గా hash చెయ్యి – float ఎప్పుడూ వద్దు." Precision bugs ను తప్పించడానికి fractions ను $(dy/g, dx/g)$ + canonical sign గా normalize చెయ్యడం – geometry problems లో పదే పదే వచ్చే idea. Vertical/horizontal line edge cases ($dx=0$ / $dy=0$) ను gcd + sign normalization సహజంగా handle చేస్తాయి.
- సాధారణ తప్పులు:
 - Slope ను float dy/dx గా వాడటం → precision వల్ల collinear points miss, $dx=0$ కి Infinity /NaN.
 - Sign ను canonical చెయ్యకపోవడం → $(1,2)$ vs $(-1,-2)$ వేరు keys, ఒకే రేఖ split అవుతుంది.
 - count + 1 (anchor కలపడం) మర్చిపోవడం → answer ఒకటి తక్కువ వస్తుంది.
 - $n \leq 2$ edge case handle చెయ్యకపోవడం (1 point → 1, కానీ loop best=1 తో మొదలైతే సరిపోతుంది; explicit guard safe).
 - Duplicate points ఉంటే (ఈ problem లో లేవు, కానీ variant లో) $dx=dy=0$ – వాటిని విడిగా లెక్కించాలి.

చివరి మాట — Signals & Cheat-sheet (అన్నీ ఒకచోట)

ఈ 14 solutions బట్టి కొట్టడం కాదు ముఖ్యం — వాటి వెనుక ఉన్న **signals** ముఖ్యం. Interview లో exact problem రాకపోవచ్చు, కానీ signal మళ్ళీ వస్తుంది. Signal కనిపిస్తే pattern గుర్తిస్తుంది; pattern ఉంటే solution వచ్చేస్తుంది.

Kadane — ఎప్పుడు?

సూచన (signal)	ఏం చెయ్యాలి	Problem
"contiguous subarray" + max/min sum	Kadane (extend vs restart)	Maximum Subarray
"circular" array + max subarray	Kadane max, మరియు $total - min$	Max Sum Circular Subarray
అన్నీ negative కావచ్చు	$maxSoFar = nums[0]$, 0 కాదు	రెండూ

Bit Manipulation — ఎప్పుడు?

సూచన (signal)	ఆయుధం	Problem
"జతలు తప్ప ఒక్కటి" (twice/one)	XOR ($a \oplus a = 0$)	Single Number
"k సార్లు తప్ప ఒక్కటి" (thrice)	bit-count % k	Single Number II
set bits లెక్కించు	$n \& (n-1)$ (Kernighan)	Number of 1 Bits
bits reverse/rearrange	shift + , చివర >>> 0	Reverse Bits
range అంతా AND	common prefix (shift దాకా)	Bitwise AND of Range
digit-by-digit + carry (base 2)	చివరి నుండి loop + carry	Add Binary

Math — ఎప్పుడు?

సూచన (signal)	Technique	Problem
digits/reverse/palindrome	% 10 , / 10	Palindrome Number
number + 1, digit array	చివరి నుండి carry	Plus One
trailing zeros / count factors	Legendre (5ల సంఖ్య)	Factorial Trailing Zeroes
"largest X such that $X^2 \leq n$ "	binary search on answer	Sqrt(x)
x^n , పెద్ద exponent	fast exponentiation ($O(\log n)$)	Pow(x, n)
collinear / slope / fraction	gcd-reduced fraction hash	Max Points on a Line

నాలుగు బంగారు నియమాలు:

- Kadane = running best.** "running sum negative అయితే throw, లేకపోతే carry; అన్నిటిలో best diary లో." Circular అయితే complement ($total - min$) ఆలోచించు.

2. **XOR = space లేకుండా జ్ఞాపకం.** జతలు cancel; జత లేనిది మిగులుతుంది. జతలు కాకపోతే bit positions ను $\% k$ చెయ్యి.
3. **$n \& (n-1)$ = lowest set bit ను తీసేయి.** Set bits count, power-of-2 check – అన్నిటికీ ఇదే.
4. **JS bitwise = 32-bit signed.** అనుమానం వస్తే $>>>$ (unsigned) వాడు, final bit-answer కి $>>> 0$. $a*b$ overflow భయమైతే $a \leq \text{limit}/b$ గా మార్పు.

గుర్తుంచుకో: ఈ మూడు categories (Kadane, Bit, Math) లో "trick" తెలిస్తే problem 5 నిమిషాల్లో, తెలియకపోతే 50 నిమిషాల్లో కూడా రాదు. కాబట్టి solutions కంటే **వాటి వెనుక ఉన్న insight** (extend-vs-restart, XOR cancel, $n \& (n-1)$), fast power, gcd-fraction) ను గుర్తుంచుకో. అదే "ఒకసారి చదివితే మర్చిపోకూడదు."